Q-learning 알고리즘이 적용된 학술논문 요약 발표

<Q-learning을 이용한 모바일 로봇의 동적 장애물 회피 및 최적 경로 탐색>

강화학습 실제
산업인공지능학과
2025254012 이은혜

1. 논문 요약 – 연구 배경과 목적

Journal of KIIT. Vol. 15, No. 9, pp. 57-62, Sep. 30, 2017. pISSN 1598-8619, eISSN 2093-7571 57
http://dx.doi.org/10.14801/jkiit.2017.15.9.57

Q-learning을 이용한 모바일 로봇의 동적 장애물 회피 및 최적 경로 탐색 알고리즘

공영훈*, 이원창**

❖ 연구 배경

- 강화학습은 Sutton의 TD 방법(Actor-Critic)과 Watkins의 Q-learning 등이 있음
- 스케줄링, 체스와 같은 게임, 로봇 제어(영상처리, 경로 추종, 군집 제어) 분야에 활용
- 기존 연구들은 대부분 모의실험 환경에만 초점을 맞춤
- 학습이 완료된 이후 동적 장애물 등장에 대응하는 방법은 제시되지 않음

❖ 연구 목적

- 모바일 로봇이 Q-learning을 통해 주변 환경을 학습하고 최적 경로를 탐색
- 동적 장애물이 존재할 때 회피할 수 있는 시스템 구현
- Tensorflow를 이용해 Q-learning 알고리즘을 구현
- 칼만 필터와 LRF를 적용하여 모바일 로봇의 위치 정확도 향상 및 장애물 인식 강화

1. 논문 요약 – 연구 진행, 연구 결과

❖ 전체 시스템 구성

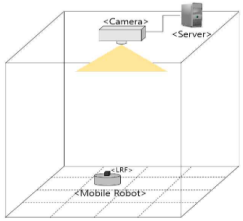


그림 1. 실험 환경
Fig. 1. Experiment environment

- 모바일 로봇의 학습 환경 : 2.4mX2.4m의 공간, 한 격자의 크기는 60cm
- 시스템 제어 : 카메라 구동, 모바일 로봇과 통신하는 서버가 전체 시스템 제어
- 모바일 로봇 : 내장된 지자기 센서로 방향을 알 수 있음. LRF를 이용해 장애물과 모바일 로봇의 거리를 측정하여 충돌 방지
- Q-learning : 모바일 로봇이 현재 위치 정보와 방향, 주변 장애물과의 거리 정보를 바탕으로 수행

❖ Q-table을 이용한 학습과 칼만필터

표 1. Q-learning 알고리즘
Table 1. Q-learning algorithm

① For each s, a initialize table entry $\hat{Q}(s,a) \leftarrow 0$
② Observe current state s
③ Do forever :
④ Select an action a and execute it
⑤ Receive immediate reward r
⑥ Observe the new state s'
⑦ Update the table entry for $\hat{Q}(s,a)$ as follows :

$$Q(s,a) \leftarrow (1-\alpha)\,Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a')\right]$$

⑧ $s \leftarrow s'$

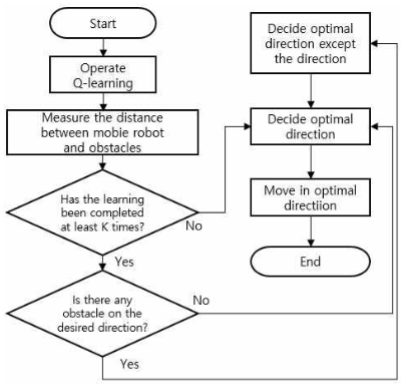


그림 2. 동적 장애물 회피 알고리즘의 처리 순서도
Fig. 2. Process sequence of avoidance algorithm for dynamic obstacle

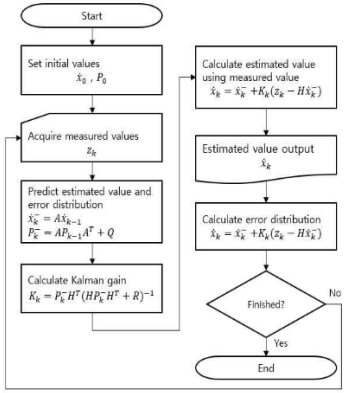


그림 3. 칼만 필터의 처리 순서도
Fig. 3. Process sequence of Kalman filter

- 모바일 로봇은 상태-행동 규칙을 표 형태(Q-table)로 저장하고, 보상 값을 통해 학습
- 일정 횟수 이상 학습 진행 후 최적 경로에 장애물이 존재하면 이 장애물이 동적 장애물로 판단됨-> 현 위치에서 장애물을 제외한 방향에서 최적경로를 판단 후 해당 방향으로 진행
- 카메라를 통해 모바일 로봇 위치를 획득하면 잡음, 측정되지 않은 부분 존재->칼만 필터를 이용하여 잡음을 제거하고 측정되지 않은 물체 위치 추정

1. 논문 요약 – 연구 진행, 연구 결과

❖ 실험 및 결과

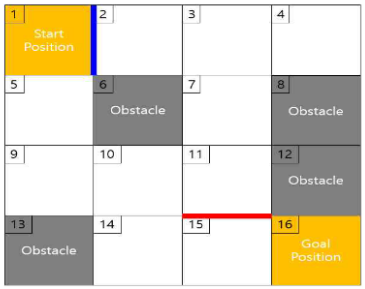


그림 4. 장애물의 위치
Fig. 4. Positions of obstacles

- 실험 환경의 특징을 포함한 표
- 1번은 시작위치, 16번은 목표 위치.
- 6, 8, 12,13번 : 장애물의 위치
- 1~2번 사이의 청색 막대, 11번~15번 사이의 적색 막대 : 동적 장애물로 가정

모바일 로봇의 이동경로 실험 결과

동적 장애물이 없을 때

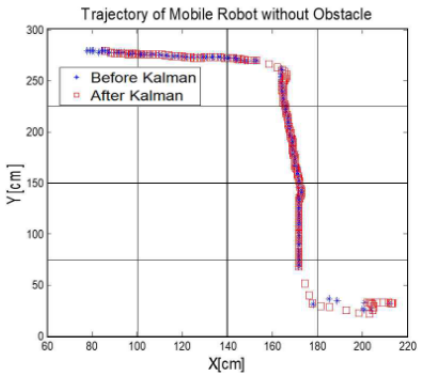


그림 5. 모바일 로봇의 이동경로
Fig. 5. Trajectory of mobile robot without dynamic obstacle

1~2번 사이의 동적 장애물이 추가되었을 때

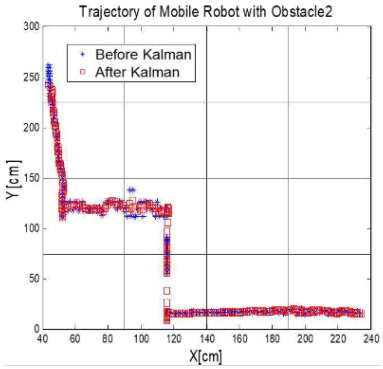


그림 7. 두 번째 동적 장애물이 있을 때 모바일 로봇의 이동경로
Fig. 7. Trajectory of mobile robot with the second dynamic obstacle

11~15번 사이의 동적 장애물이 추가되었을 때

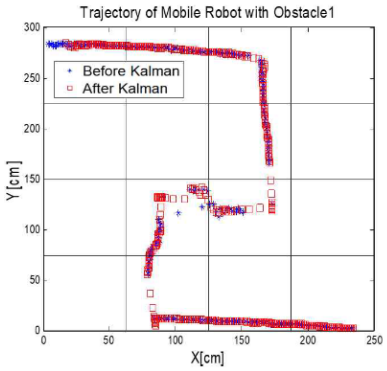


그림 6. 첫 번째 동적 장애물이 있을 때 모바일 로봇의 이동경로
Fig. 6. Trajectory of mobile robot with the fisrt dynamic obstacle

2. 강화학습의 기본과 구성요소

❖ 강화학습 기본 구조

- Agent가 Environment와 상호작용하면서 보상을 최대화하는 방향으로 학습
- 핵심 목표: 장기적인 총 보상 기대값을 최대화
- 기본 과정: 상태 관찰 → 행동 선택 → 보상 수신 → 상태 업데이트 → 학습

❖ 강화학습 구성 요소와 논문 내 값

	Agent	Environment	State	Action	Reward	Policy	Value Function
정의	행동을 결정하는 주체	Agent가 상호작용하는 공간	현재 환경의 정보	에이전트가 선택할 수 있는 행동	행동의 결과로 받는 보상	Agent가 어떤 Action을 선택하는지 결정하는 전략	상태나 상태-행동 쌍의 가치를 평가
논문	모바일로봇	실험 공간 (4x4격자)	로봇의 위치	상,하,좌,우 이동	이동 결과를 통한 간접적 보상 (목표 접근 성공/실패)	Decaying ε-greedy	Q-table의 Q(s,a) 값을 통해 상태-행동쌍의 기대 보상 추정

3. Q-learning의 기본 원리와 논문 내 적용

❖ Q-learning의 기본 원리

- Q-learning은 미래의 기대 보상을 최대화하기 위해 상태(state)와 행동(action) 쌍의 가치를 학습하는 강화학습 방법
- 자신과 다른 환경에서 쌓은 경험으로 자신의 정책을 개선하는 Off-policy 기법으로 대상 정책과 행동 정책이 다름
    - 에이전트는 자신의 정책과 다른 행동 정책(ε-greedy)을 통해 탐험하며 데이터를 수집
    - 수집한 데이터는 현재 Q값과 미래 Q값(max Q(s', a'))를 이용하여 업데이트
    - -> 행동은 ε-greedy로 탐험하지만 학습은 최적 행동 기준으로 수행하는 off-policy 기법
- 한 번의 행동 이후 바로 Q값을 업데이트(TD(Temporal Difference) 방식)
- 별도의 중요도 샘플링(importance sampling) 없이 실제 관찰한 보상과 다음 상태만으로 학습을 진행
- 최종적으로 Q-table을 통해 상태-행동 쌍의 기대 보상을 학습하고 이를 기반으로 최적 정책을 찾음.

$$Q'(S_t, A_t) = Q(S_t, A_t) + \alpha\left\{R_t + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right\}$$

1. $Q(s,a)$: 현재 상태 s에서 행동 a를 선택했을 때 기대되는 보상
2. α (학습률) : 새로 얻은 정보가 기존 Q값에 얼마나 반영될지 결정
3. r (보상) : 행동 후 즉시 받은 보상
4. γ (할인율) : 미래 보상의 현재 가치에 대한 중요도
5. max$a'Q(s',a')$: 다음 상태 s'에서 가능한 행동 중 최대 Q값

3. Q-learning의 기본 원리와 논문 내 적용

❖ Q-learning의 논문 내 적용

표 1. Q-learning 알고리즘
Table 1. Q-learning algorithm

① For each s, a initialize table entry $\hat{Q}(s,a) \leftarrow 0$
② Observe current state s
③ Do forever :
④ Select an action a and execute it
⑤ Receive immediate reward r
⑥ Observe the new state s'
⑦ Update the table entry for $\hat{Q}(s,a)$ as follows :

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha [\, r + \gamma \max_{a'} Q(s',a')]$$

⑧ $s \leftarrow s'$

	정의	논문
정책평가(Policy Evaluation)	정책이 주어졌을 때 그 정책의 가치 함수를 구하는 문제	Q-Table 업데이트
정책 제어 (Policy control)	정책을 조정하여 최적 정책을 만들어낸 것	Decaying ε-greedy 방법으로 정책을 점진적으로 개선함

학습 흐름

1. 초기 설정 (①~③)
   - 모든 상태-행동 쌍 (s,a)에 대해 Q값을 0으로 초기화
   - State s: 모바일 로봇의 현재 위치
   - Action a: 상, 하, 좌, 우 방향 이동

Policy control

2. 행동 선택 (④)
   - Decaying ε-greedy 기법을 적용하여 행동 선택
3. 보상 및 다음 상태 관찰 (⑤~⑥)
   - 행동 실행 후 보상 받음
   - 새로운 상태 s' 관찰

Policy Evaluation

4. Q-table 업데이트 (⑦)
   - 보상과 다음 상태를 기반으로 Q-table을 업데이트
   - 기대 보상 반영
   - 업데이트 식에 할인계수 γ를 적용해 미래 보상의 현재 가치를 반영
5. 환경 특성
   - 실험 환경은 예기치 못한 오차가 존재하는 확률적 모델
   - 불규칙한 잡음의 영향을 줄이기 위해 학습률 α를 조정하여 안정적인 학습을 유도
6. 학습 반복 (⑧)
   - 다음 상태 s' 를 현재 상태로 갱신
   - ④~⑧ 과정을 반복하여 최적 경로를 학습

❖ 동적 장애물 회피 알고리즘 추가

- Q-learning으로 기본 최적 경로를 학습한 후 실시간으로 추가된 동적 장애물에 대응하는 알고리즘 적용
- 목표: 이동 중 장애물이 감지되면 장애물 방향을 제외한 다른 방향 중 최적 경로를 재선택하여 이동

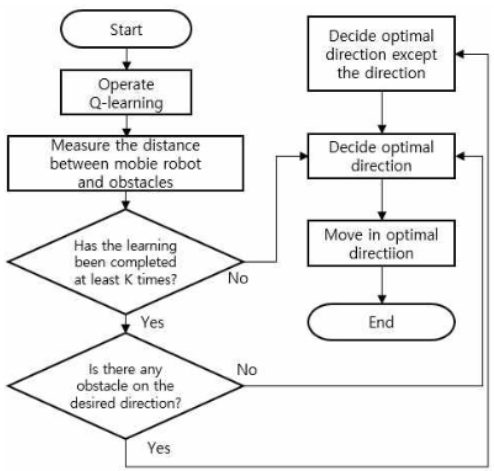


그림 2. 동적 장애물 회피 알고리즘의 처리 순서도
Fig. 2. Process sequence of avoidance algorithm for dynamic obstacle

동적 장애물 회피 알고리즘

1. Q-learning 기반 경로 이동
2. 장애물 감지
   - 이동하려는 방향에 장애물이 존재하는지 측정
3. 장애물 존재 여부 판단
   - 장애물이 없다면 최적 경로로 이동
   - 장애물이 있으면 해당 방향 제외
4. 대체 경로 결정
   - 남은 방향 중 Q값이 가장 높은 방향 선택
5. 대체 경로로 이동
   - 수정된 최적 경로를 따라 이동

4. 실험

실험 순서

1. 4X4, 4개의 고정 장애물이 있는 Gridworld 생성
2. 임의의 α, γ, ε 값을 설정하여 q-learning 진행
3. 기본 고정 장애물이 있는 경우의 최적 경로 확인
4. Q-table 결과 값에 동적 장애물이 추가된 경우 회피 알고리즘 적용
5. 11~15, 1~2번 사이의 장애물이 추가되었을 때 최적 경로 확인

❖ 모바일 로봇의 Environment – 4x4 격자

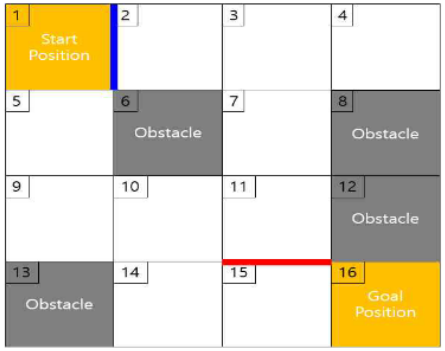
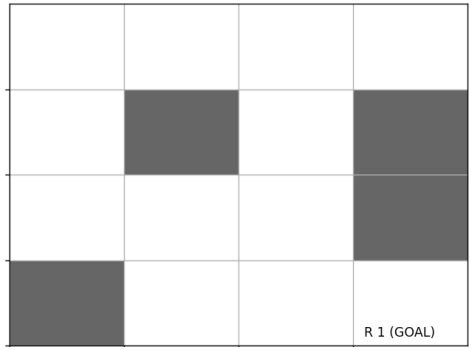


그림 4. 장애물의 위치
Fig. 4. Positions of obstacles

4x4 격자와 고정 장애물(GridWorld 활용)



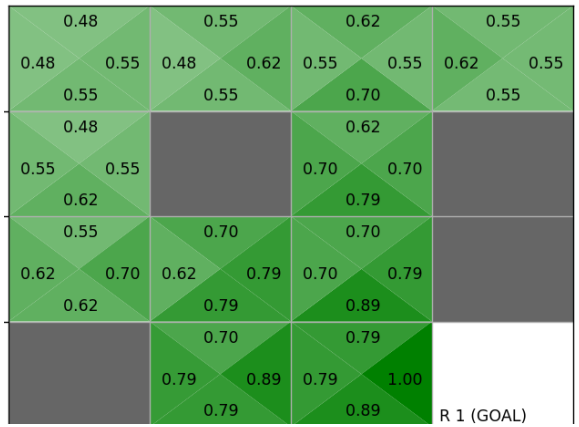
4. 실험

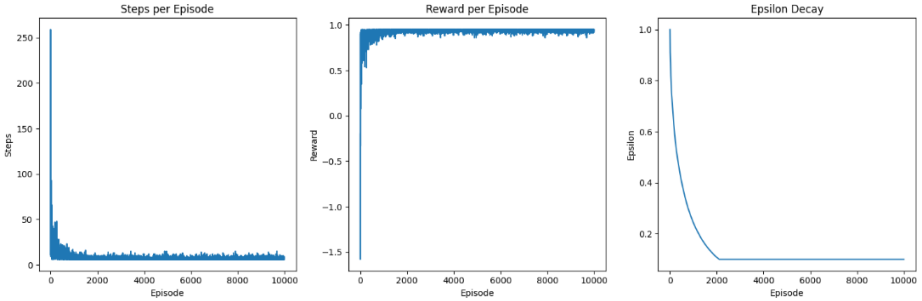
❖ Q-learning 결과

* 논문에 수치가 언급되지 않아 임의 값으로 실험 진행

1. α (학습률) = 0.8
2. γ (할인율) = 0.9
3. Decaying ε-greedy로 처음 ε값을 1로 하고 학습을 진행할 수록 0.0001씩 감소하도록 설정, 최소값은 0.1로 설정
4. 에피소드 10000번 학습

고정 장애물 위치가 있는 Q-table





- Steps per Episode : 학습 초반엔 스탭이 많다가 점점 줄어들어 최적 경로를 빠르게 찾음
- Reward per Episode : 초반에는 마이너스가 많았다가 최종 보상 1 근처로 수렴
- Epsilon Decay : 처음엔 ε 1.0으로 탐험 위주였다가 점점 감소하여 0.1로 안정화

4. 실험

❖ Case마다의 최적 경로

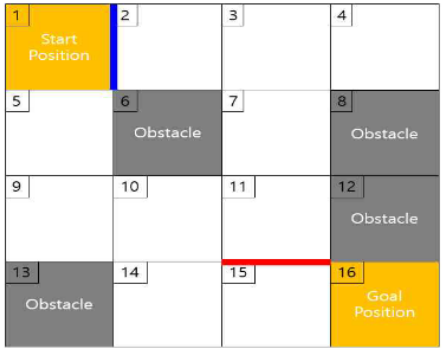
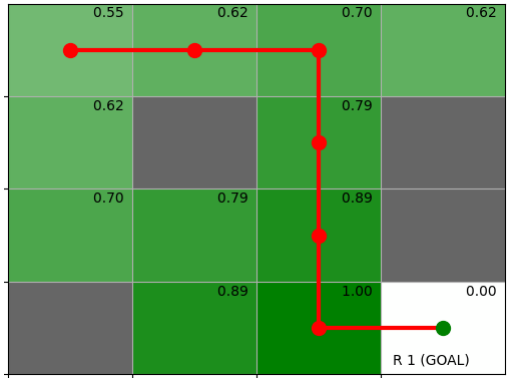


그림 4. 장애물의 위치
Fig. 4. Positions of obstacles

- Case1 : 고정 장애물만 있는 경우
- Case 2: 1->2 경로가 막힐 때
  파란색 장애물이 추가된 경우
- Case 3: 11->15 경로가 막힐 때
  빨간색 장애물이 추가된 경우

*Case2~3 : 동적 장애물 회피 알고리즘 구현하여 적용

Case0



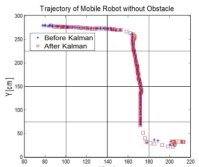
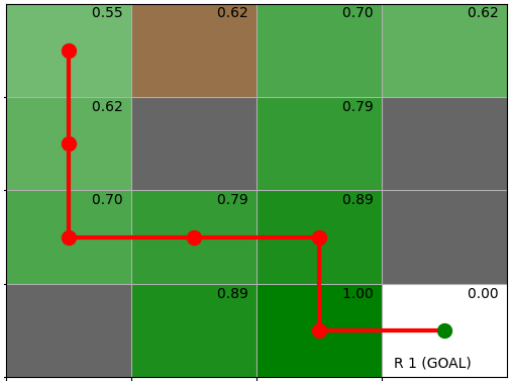


그림 5. 모바일 로봇의 이동경로
Fig. 5. Trajectory of mobile robot without dynamic obstacle

Case1



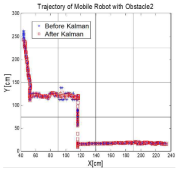
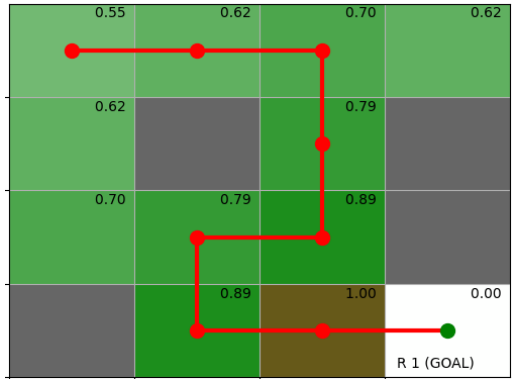


그림 7. 두 번째 동적 장애물이 있을 때 모바일 로봇의 이동경로
Fig. 7. Trajectory of mobile robot with the second dynamic obstacle

Case2



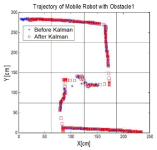


그림 6. 첫 번째 동적 장애물이 있을 때 모바일 로봇의 이동경로
Fig. 6. Trajectory of mobile robot with the fisrt dynamic obstacle

5. 주요 구현 코드

❖ Agent

```python
class QLearningAgent:  1 usage
    def __init__(self):
        self.gamma = 0.9
        self.alpha = 0.8  # 학습률을 다시 0.8로
        self.epsilon_start = 1.0
        self.epsilon_end = 0.1
        self.epsilon_decay = 0.0001
        self.epsilon = self.epsilon_start
        self.action_size = 4
        self.Q = defaultdict(lambda: 0)

    def get_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.action_size)
        else:
            qs = [self.Q[state, a] for a in range(self.action_size)]
            max_q = max(qs)
            # 최대 Q값을 가진 행동들을 모두 찾아서 그 중에서 랜덤하게 선택
            max_actions = [a for a, q in enumerate(qs) if q == max_q]
            return np.random.choice(max_actions)

    def update(self, state, action, reward, next_state, done):
        if done:
            next_q_max = 0
        else:
            next_qs = [self.Q[next_state, a] for a in range(self.action_size)]
            next_q_max = max(next_qs)

        target = reward + self.gamma * next_q_max
        self.Q[state, action] += self.alpha * (target - self.Q[state, action])

        # 입실론 값 감소
        self.epsilon = max(
            self.epsilon_end,
            self.epsilon * (1 - self.epsilon_decay)  # 지수적 감소로 변경
        )
```

❖ Q-learning

```python
if __name__ == '__main__':
    env = FourGridWorld()
    agent = QLearningAgent()

    # 기본 그리드월드 시각화 (장애물만)
    print("\n=== 기본 그리드월드 ===")
    env.render_v()

    # Q-table 학습
    episodes = 10000
    epsilon_history = []
    steps_history = []
    rewards_history = []

    print("=== Q-learning 학습 시작 ===")
    for episode in range(episodes):
        state = env.reset()
        epsilon_history.append(agent.epsilon)
        total_reward = 0
        steps = 0

        while True:
            action = agent.get_action(state)
            next_state, reward, done = env.step(action)
            agent.update(state, action, reward, next_state, done)
            total_reward += reward
            steps += 1

            if done or steps >= env.max_steps:  # 최대 스텝 수 제한 추가
                steps_history.append(steps)
                rewards_history.append(total_reward)
                break
            state = next_state

        # 입실론 감소
        agent.epsilon = max(
            agent.epsilon_end,
            agent.epsilon * (1 - agent.epsilon_decay)  # 지수적 감소로 변경
        )
```

5. 주요 구현 코드

❖ 동적 장애물 회피 알고리즘

```python
def find_path_with_dynamic_walls(self, Q):  3 usages
    """동적 장애물을 피해 경로 찾기"""
    path = []
    current_state = self.start_state
    visited = set()  # 방문한 상태 기록

    while current_state != self.goal_state:
        if current_state in visited:  # 순환 경로 방지
            return None

        visited.add(current_state)
        path.append(current_state)

        # 현재 상태에서 가능한 모든 행동의 Q값 계산
        q_values = []
        available_actions = []

        for action in self.action_space:
            next_state = self.next_state(current_state, action)
            if next_state != current_state:  # 이동 가능한 경우만
                q_values.append(Q.get((current_state, action), 0))
                available_actions.append(action)

        if not available_actions:
            return None  # 더 이상 이동할 수 없음

        # Q값이 높은 순서대로 행동 정렬
        sorted_indices = np.argsort(q_values)[::-1]

        # Q값이 높은 행동부터 시도
        moved = False
        for idx in sorted_indices:
            action = available_actions[idx]
            next_state = self.next_state(current_state, action)

            if next_state != current_state and next_state not in visited:
                current_state = next_state
                moved = True
                break

        if not moved:
            return None  # 더 이상 진행할 수 없음

    path.append(self.goal_state)
    return path
```

감사합니다.